Estación Meteorológica IoT

Implementación de una solución con Apache Kafka

CC3067 Redes | Universidad del Valle de Guatemala

Roberto Barreda (23354) | Javier España (23361)

Estación meteorológica

Objetivo y arquitectura

Se implementó una estación meteorológica IoT simulada. Un productor genera telemetría, la publica en el broker edge y un consumidor la recupera para desplegarla en tiempo real

- Broker Kafka edge: 161.35.11.141:9092, topic único: 23361
- Producer: temperatura, humedad y dirección del viento
- Consumer: series temporales y rosa de vientos
- Payload binario compacto de exactamente 3 bytes

```
Producer -> Kafka topic 23361 -> Consumer
   JSON  |  Binario de 3 bytes
   SASL_PLAINTEXT / PLAIN
```

Patrón publish/subscribe

Simulación del sensor

El nodo simulado genera una lectura con tres sensores. Temperatura y humedad usan una distribución normal centrada en 55. El viento se selecciona uniformemente entre ocho direcciones

- Temperatura: 0 a 110.00 °C, dos decimales, media 55 y desviación 15
- Humedad: 0 a 100 %, entero, media 55 y desviación 15
- Viento: N, NE, E, SE, S, SO, O y NO
- Estructura JSON: temperatura, humedad y dirección_viento

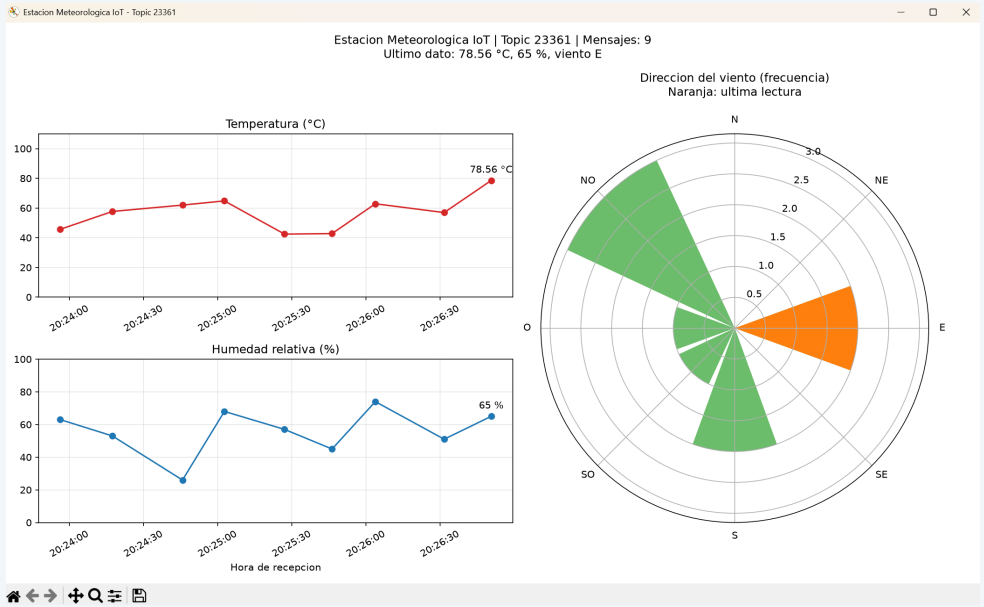
```
{"temperatura": 56.32,
 "humedad": 51,
 "direccion_viento": "SO"}
```

Representación legible y portable

Envío al servidor edge

El productor se conecta al broker mediante confluent-kafka, publica en el topic 23361 usando la key sensor1 y permanece activo hasta Ctrl+C. El intervalo normal entre lecturas es de 15 a 30 segundos

- bootstrap.servers: 161.35.11.141:9092
- security.protocol: SASL_PLAINTEXT
- sasl.mechanisms: PLAIN
- El callback reporta partición y offset de entrega

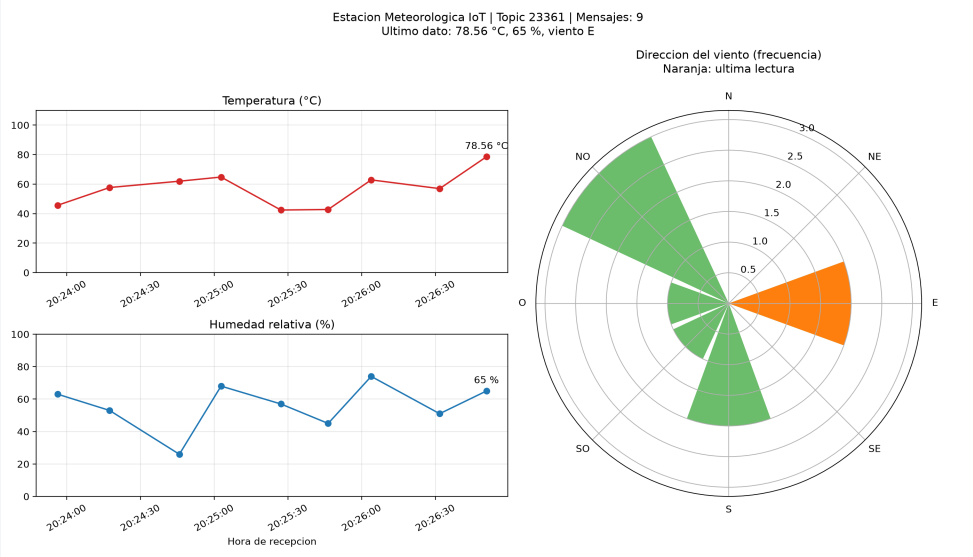


Captura de la telemetria generada

Consumo y despliegue

El consumidor usa un group.id propio y auto.offset.reset=earliest. Cada mensaje se decodifica, se agrega al historial y actualiza la figura

- Gráfica de temperatura con sus valores en el tiempo
- Gráfica de humedad relativa con sus valores en el tiempo
- Rosa de vientos con frecuencias por dirección
- La última dirección se resalta en naranja



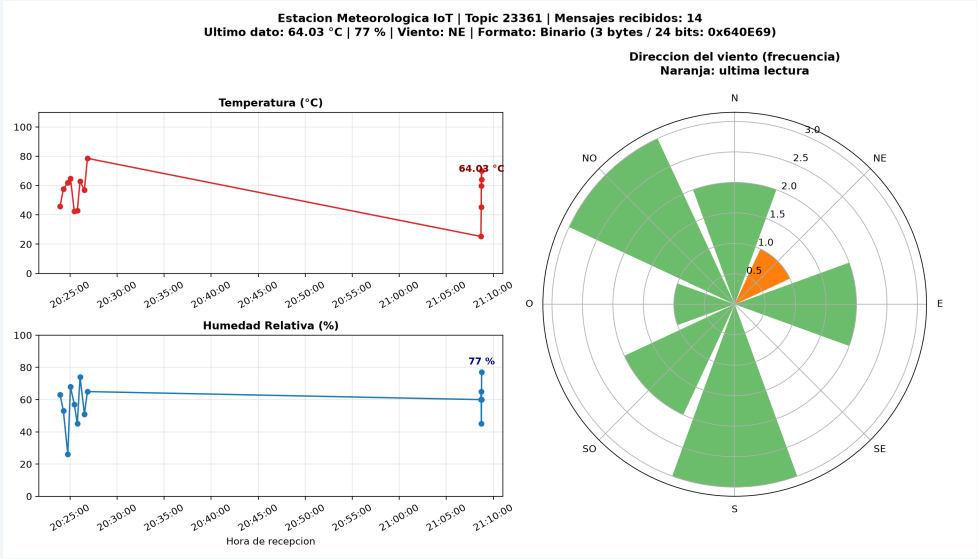
Captura del consumidor con nueve mensajes procesados

Restricción del payload

Para cumplir el máximo de 3 bytes se eliminó el texto del JSON y se empaquetaron los campos en un entero de 24 bits. La misma función se usa para codificar y decodificar en producer y consumer

```
24 bits = [ temperatura 14 ] [ humedad 7 ] [ viento 3 ]
temp_int = round(temperatura * 100)
valor = (temp_int << 10) | (humedad << 3) | viento
```

- Temperatura: 0..11000 centésimas, 2^14 - 1 = 16383
- Humedad: 0..100, 7 bits permiten hasta 127
- Viento: 8 categorías, 3 bits permiten exactamente 8 valores



Captura del consumidor con catorce mensajes binarios

Evidencia del codec y pruebas

El módulo codec.py implementa encode_payload/decode_payload, además de json_to_bytes/bytes_to_json. Las pruebas cubren límites, direcciones y la inspección visual de bits

```
python -m unittest -v test_codec.py
Ran 6 tests in 0.001s
OK
```

- Payload de salida siempre: len(payload) == 3
- Los límites 0.00/110.00 y 0/100 son válidos
- Las ocho direcciones se codifican y recuperan
- La gráfica confirma que el consumidor decodifica el formato

Decisiones de formato y arquitectura

JSON/SOAP y OSI

Se ubican conceptualmente en la capa 6, presentación, porque serializan y estructuran datos. En TCP/IP se consideran parte de la capa de aplicación. Sus beneficios son interoperabilidad, legibilidad, flexibilidad de esquema y amplio soporte de librerías

Pub/Sub con Kafka

Ventajas: desacoplamiento, escalabilidad por particiones y consumer groups, persistencia y reprocesamiento. Desventajas: infraestructura pesada, overhead y poca conveniencia para microcontroladores muy restringidos

Casos de uso

Adecuado para streaming masivo, analítica en tiempo real, event sourcing y logs. No es ideal para mensajes punto a punto simples, CRUD de baja escala o hard real-time

Diseño del payload compacto

Complejidades

Un payload pequeño elimina nombres de campos, exige un contrato de bits, cuantización, validación estricta y versionado. Un bit corrupto puede afectar varios valores

Temperatura en 14 bits

Se multiplica por 100: 0.00..110.00 se convierte en 0..11000. Como 2^14 - 1 = 16383, el rango completo cabe sin perder los dos decimales

Si humedad fuera float con un decimal

Habría que escalarla por 10: 0..1000 requiere 10 bits. El presupuesto 14 + 10 + 3 = 27 bits excede 24. Habría que reducir precisión, rango, enviar dos paquetes o permitir un payload mayor

Optimización y conclusiones

Kafka puede ayudar con compression.type (zstd, lz4, snappy o gzip), batching con linger.ms y batch.size, serialización binaria Avro/Protobuf con Schema Registry, topics compactados y un gateway MQTT/CoAP hacia Kafka

Resultado final

- Se simularon los sensores y se envió telemetría al broker edge
- Se consumieron y graficaron los datos en tiempo real
- Se repitió el flujo usando un payload binario de 3 bytes
- La evidencia visual y las respuestas de la guía quedan reunidas en esta presentación

Archivos relacionados: producer.py, consumer.py, codec.py y test_codec.py